

Understanding the Dataset

The dataset has a list of all the TV shows/movies available on Netflix:

Show_id: Unique ID for every Movie / Tv Show

Type: Identifier - A Movie or TV Show

Title: Title of the Movie / Tv Show

Director: Director of the Movie

Cast: Actors involved in the movie/show

Country: Country where the movie/show was produced

Date_added: Date it was added on Netflix

Release_year: Actual Release year of the movie/show

Rating: Maturity Rating of the movie/show

Duration: Total Duration - in minutes or number of seasons

Listed_in: Genre

Description: The summary description

Importing all the necessary libraries

```
In [87]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
#from wordcloud import WordCloud
```

Reading and checking the dataset

```
In [88]: df = pd.read_csv("netflix.csv")
df.head()

#any column having more than 50% unique values
#that column never contribute in project
```

Out[88]:

	show_id	type	title	director	cast	country	date_added	release_year	rating
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2020	PG-
1	s2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	September 24, 2021	2021	T M
2	s3	TV Show	Ganglands	Julien Leclercq	Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...	NaN	September 24, 2021	2021	T M
3	s4	TV Show	Jailbirds New Orleans	NaN	NaN	NaN	September 24, 2021	2021	T M
4	s5	TV Show	Kota Factory	NaN	Mayur More, Jitendra Kumar, Ranjan Raj, Alam K...	India	September 24, 2021	2021	T M

In [89]: `df.columns`

Out[89]: Index(['show_id', 'type', 'title', 'director', 'cast', 'country', 'date_added', 'release_year', 'rating', 'duration', 'listed_in', 'description'], dtype='object')

Renaming the listed in column to genres

In [90]: `df.rename(columns={'listed_in': 'genre'}, inplace=True)`
`df.columns`

Out[90]: Index(['show_id', 'type', 'title', 'director', 'cast', 'country', 'date_added', 'release_year', 'rating', 'duration', 'genre', 'description'], dtype='object')

Dropping the show_id column as it may be of no use in the analysis

```
In [91]: df.drop(columns='show_id',inplace=True)
```

```
In [92]: #identical rows---means by mistake you enter same data for so many time it also c  
#the performance of analysis
```

Checking for duplicates

```
In [93]: df.duplicated().sum()
```

```
Out[93]: np.int64(0)
```

Checking the missing values

```
In [94]: df.isnull().sum()
```

```
Out[94]: type          0  
title          0  
director      2634  
cast          825  
country       831  
date_added     10  
release_year   0  
rating         4  
duration       3  
genre          0  
description    0  
dtype: int64
```

```
In [95]: #here we can see clearly here some count are so big and some are so small  
#which is available in less quantity we can drop them
```

```
In [96]: #as we have studied  
#if we write df.dropna()---all the rows having missing values got deleted  
# if we write df.dropna(axis=1) the columns having missing value got deleted  
  
#if we want to delete missing value from specific column  
#df.dropna(subset=["Column_name"],axis=1,inplace=True)
```

```
In [97]: df.shape
```

```
Out[97]: (8807, 11)
```

Dropping the rows which have null values in date_added column as there are only 10 values which sums up to only 0.11% of the total values

```
In [98]: df.dropna(subset = ['date_added'],inplace=True)  
df['date_added'].isna().sum()
```

```
Out[98]: np.int64(0)
```

Dropping the rows which have null values in rating column as there are only 4 values which sums up to only 0.05% of the total values

```
In [99]: df.dropna(subset = ['rating'], inplace=True)
df['rating'].isna().sum()
```

```
Out[99]: np.int64(0)
```

Dropping the rows which have null values in duration column as there are only 3 values which sums up to only 0.03% of the total values

```
In [100... df.dropna(subset = ['duration'], inplace=True)
df['duration'].isna().sum()
```

```
Out[100... np.int64(0)
```

```
In [101... df.isnull().sum()
```

```
Out[101... type          0
title          0
director      2621
cast          825
country       829
date_added    0
release_year  0
rating        0
duration      0
genre         0
description   0
dtype: int64
```

```
In [102... df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 8790 entries, 0 to 8806
Data columns (total 11 columns):
#   Column          Non-Null Count  Dtype
---  -
0   type            8790 non-null  object
1   title           8790 non-null  object
2   director        6169 non-null  object
3   cast            7965 non-null  object
4   country         7961 non-null  object
5   date_added      8790 non-null  object
6   release_year    8790 non-null  int64
7   rating          8790 non-null  object
8   duration        8790 non-null  object
9   genre           8790 non-null  object
10  description      8790 non-null  object
dtypes: int64(1), object(10)
memory usage: 824.1+ KB
```

Converting the date added column from object to datetime format

```
In [103... #when there is some mismatch in our date format then that value is replaced by null

df['date_added']=pd.to_datetime(df['date_added'],errors='coerce')
df['date_added'].isnull().sum()
```

```
Out[103... np.int64(88)
```

```
In [104... df.dropna(subset = ['date_added'],inplace=True)
df['date_added'].isna().sum()
```

```
Out[104... np.int64(0)
```

Adding day, month, year, month_name, day_name as seperate columns to the dataframe as these will help us in analysis

```
In [105... df['day'] = df['date_added'].dt.day
df['month'] = df['date_added'].dt.month
df['year'] = df['date_added'].dt.year
df['month_name'] = df['date_added'].dt.month_name()
df['weekday'] = df['date_added'].dt.day_name()
df.head()
```

Out[105...

	type	title	director	cast	country	date_added	release_year	rating	durat
0	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	2021-09-25	2020	PG-13	90 r
1	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	2021-09-24	2021	TV-MA	Seas
2	TV Show	Ganglands	Julien Leclercq	Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...	NaN	2021-09-24	2021	TV-MA	1 Seas
3	TV Show	Jailbirds New Orleans	NaN	NaN	NaN	2021-09-24	2021	TV-MA	1 Seas
4	TV Show	Kota Factory	NaN	Mayur More, Jitendra Kumar, Ranjan Raj, Alam K...	India	2021-09-24	2021	TV-MA	Seas

In [106...

```
df["duration"].unique()
```

```
Out[106... array(['90 min', '2 Seasons', '1 Season', '91 min', '125 min',
      '9 Seasons', '104 min', '127 min', '4 Seasons', '67 min', '94 min',
      '5 Seasons', '161 min', '61 min', '166 min', '147 min', '103 min',
      '97 min', '106 min', '111 min', '3 Seasons', '110 min', '105 min',
      '96 min', '124 min', '116 min', '98 min', '23 min', '115 min',
      '122 min', '99 min', '88 min', '100 min', '6 Seasons', '102 min',
      '93 min', '95 min', '85 min', '83 min', '113 min', '13 min',
      '182 min', '48 min', '145 min', '87 min', '92 min', '80 min',
      '117 min', '128 min', '119 min', '143 min', '114 min', '118 min',
      '108 min', '63 min', '121 min', '142 min', '154 min', '120 min',
      '82 min', '109 min', '101 min', '86 min', '229 min', '76 min',
      '89 min', '156 min', '112 min', '107 min', '129 min', '135 min',
      '136 min', '165 min', '150 min', '133 min', '70 min', '84 min',
      '140 min', '78 min', '7 Seasons', '64 min', '59 min', '139 min',
      '69 min', '148 min', '189 min', '141 min', '130 min', '138 min',
      '81 min', '132 min', '10 Seasons', '123 min', '65 min', '68 min',
      '66 min', '62 min', '74 min', '131 min', '39 min', '46 min',
      '38 min', '8 Seasons', '17 Seasons', '126 min', '155 min',
      '159 min', '137 min', '12 min', '273 min', '36 min', '34 min',
      '77 min', '60 min', '49 min', '58 min', '72 min', '204 min',
      '212 min', '25 min', '73 min', '29 min', '47 min', '32 min',
      '35 min', '71 min', '149 min', '33 min', '15 min', '54 min',
      '224 min', '162 min', '37 min', '75 min', '79 min', '55 min',
      '158 min', '164 min', '173 min', '181 min', '185 min', '21 min',
      '24 min', '51 min', '151 min', '42 min', '22 min', '134 min',
      '177 min', '13 Seasons', '52 min', '14 min', '53 min', '8 min',
      '57 min', '28 min', '50 min', '9 min', '26 min', '45 min',
      '171 min', '27 min', '44 min', '146 min', '20 min', '157 min',
      '17 min', '203 min', '41 min', '30 min', '194 min', '15 Seasons',
      '233 min', '237 min', '230 min', '195 min', '253 min', '152 min',
      '190 min', '160 min', '208 min', '180 min', '144 min', '5 min',
      '174 min', '170 min', '192 min', '209 min', '187 min', '172 min',
      '16 min', '186 min', '11 min', '193 min', '176 min', '56 min',
      '169 min', '40 min', '10 min', '3 min', '168 min', '312 min',
      '153 min', '214 min', '31 min', '163 min', '19 min', '12 Seasons',
      '179 min', '43 min', '200 min', '196 min', '167 min', '178 min',
      '228 min', '18 min', '205 min', '201 min', '191 min'], dtype=object)
```

```
In [107... s1='179 min'
            int(s1.split()[0])
```

```
Out[107... 179
```

```
In [108... s1='12 Seasons'
            int(s1.split()[0])
```

```
Out[108... 12
```

```
In [109... df["duration"]=df["duration"].apply(lambda x:int(x.split()[0]))
```

```
In [110... df["duration"].unique()
```

```
Out[110...] array([ 90,  2,  1, 91, 125,  9, 104, 127,  4, 67, 94,  5, 161,
        61, 166, 147, 103, 97, 106, 111,  3, 110, 105, 96, 124, 116,
        98, 23, 115, 122, 99, 88, 100,  6, 102, 93, 95, 85, 83,
        113, 13, 182, 48, 145, 87, 92, 80, 117, 128, 119, 143, 114,
        118, 108, 63, 121, 142, 154, 120, 82, 109, 101, 86, 229, 76,
        89, 156, 112, 107, 129, 135, 136, 165, 150, 133, 70, 84, 140,
        78,  7, 64, 59, 139, 69, 148, 189, 141, 130, 138, 81, 132,
        10, 123, 65, 68, 66, 62, 74, 131, 39, 46, 38,  8, 17,
        126, 155, 159, 137, 12, 273, 36, 34, 77, 60, 49, 58, 72,
        204, 212, 25, 73, 29, 47, 32, 35, 71, 149, 33, 15, 54,
        224, 162, 37, 75, 79, 55, 158, 164, 173, 181, 185, 21, 24,
        51, 151, 42, 22, 134, 177, 52, 14, 53, 57, 28, 50, 26,
        45, 171, 27, 44, 146, 20, 157, 203, 41, 30, 194, 233, 237,
        230, 195, 253, 152, 190, 160, 208, 180, 144, 174, 170, 192, 209,
        187, 172, 16, 186, 11, 193, 176, 56, 169, 40, 168, 312, 153,
        214, 31, 163, 19, 179, 43, 200, 196, 167, 178, 228, 18, 205,
        201, 191])
```

Checking for unique values of rating and categorising them for better understanding

```
In [111...] df['rating'].unique()
```

```
Out[111...] array(['PG-13', 'TV-MA', 'PG', 'TV-14', 'TV-PG', 'TV-Y', 'TV-Y7', 'R',
        'TV-G', 'G', 'NC-17', 'NR', 'TV-Y7-FV', 'UR'], dtype=object)
```

Netflix has officially categorised these into 3 main categories

(<https://help.netflix.com/en/node/2064/us>), so using that for our EDA

Replacing 'TV-Y','TV-Y7','G','TV-G','PG','TV-PG','TV-Y7-FV' to Kids,

'PG-13','TV-14' to Teens,

'R','TV-MA','NC-17' to Adults,

and 'NR','UR' to null values

```
In [112...] df['rating'] = df['rating'].replace(['TV-Y', 'TV-Y7', 'G', 'TV-G', 'PG', 'TV-PG', 'TV-Y7-FV',
df['rating'] = df['rating'].replace(['PG-13', 'TV-14'], 'Teens')
df['rating'] = df['rating'].replace(['R', 'TV-MA', 'NC-17'], 'Adults')
df['rating'] = df['rating'].replace(['NR', 'UR'], np.nan)
df.head()
```


Out[112...

	type	title	director	cast	country	date_added	release_year	rating	durat
0	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	2021-09-25	2020	Teens	
1	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	2021-09-24	2021	Adults	
2	TV Show	Ganglands	Julien Leclercq	Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...	NaN	2021-09-24	2021	Adults	
3	TV Show	Jailbirds New Orleans	NaN	NaN	NaN	2021-09-24	2021	Adults	
4	TV Show	Kota Factory	NaN	Mayur More, Jitendra Kumar, Ranjan Raj, Alam K...	India	2021-09-24	2021	Adults	



In []:

In [113...

```
df['rating'].unique()
```

Out[113...

```
array(['Teens', 'Adults', 'Kids', nan], dtype=object)
```

From here it is confirmed that values have been replaced successfully and now we will delete rows that have null values.

In [114...

```
df['rating'].isna().sum()
```

Out[114...

```
np.int64(81)
```

```
In [115... df.dropna(subset = ['rating'],inplace=True)
df['rating'].isna().sum()
```

```
Out[115... np.int64(0)
```

82 values were null, which was only about 1% of the data, so we deleted those rows.

```
In [116... df.isna().sum()
```

```
Out[116... type          0
title          0
director      2535
cast          800
country       826
date_added    0
release_year  0
rating        0
duration      0
genre         0
description   0
day           0
month         0
year          0
month_name    0
weekday       0
dtype: int64
```

Now, we still have about 48% missing data. So, we will check in which rows the director, country and cast all three are null, if all these 3 are null then there is no use of that row.

```
In [117... df[(df['director'].isna()) & (df['country'].isna()) & (df['cast'].isna())].shape[0]
```

```
Out[117... 96
```

We found out that total 96 such rows are there which dont have director, country and cast as null, so deleting those rows.

```
In [118... df.shape
```

```
Out[118... (8621, 16)
```

```
In [119... df.dropna(subset = ['director','country','cast'], how = 'all',inplace=True)
df.shape
```

```
Out[119... (8525, 16)
```

```
In [120... df.isna().sum()
```

```
Out[120...] type          0
          title          0
          director    2439
          cast        704
          country     730
          date_added   0
          release_year 0
          rating       0
          duration     0
          genre        0
          description   0
          day          0
          month        0
          year         0
          month_name    0
          weekday      0
          dtype: int64
```

```
In [121...] df.isna().sum()
```

```
Out[121...] type          0
          title          0
          director    2439
          cast        704
          country     730
          date_added   0
          release_year 0
          rating       0
          duration     0
          genre        0
          description   0
          day          0
          month        0
          year         0
          month_name    0
          weekday      0
          dtype: int64
```

As the missing value percentage of these rows are significant, we cant delete them so we will deal with those later and now lets focus

on one more major problem in the dataset which is NESTED DATA in director, cast, country, and genre.

```
In [122...] df["cast"][2].split(", ")
```

```
Out[122...] ['Sami Bouajila',
              'Tracy Gotoas',
              'Samuel Jouy',
              'Nabiha Akkari',
              'Sofia Lesaffre',
              'Salim Kechiouche',
              'Noureddine Farihi',
              'Geert Van Rampelberg',
              'Bakary Diombera']
```

```
In [123...] df_cast = pd.DataFrame(df['cast'].apply(lambda x: str(x).split(', ')).tolist(), index=df_cast)
```

```
Out[123...]
           0           1           2           3           4           5           6
title
Dick Johnson Is Dead      nan      None      None      None      None      None      None
Blood & Water            Ama      Khosi      Gail      Thabang      Dillon      Natasha      Arno Greeft
                        Qamata      Ngema      Mabalane      Molaba      Windvogel      Thahane
Ganglands                Sami      Tracy      Samuel      Nabiha      Sofia      Salim      Noureddine
                        Bouajila      Gotoas      Jouy      Akkari      Lesaffre      Kechiouche      Farihi
Kota Factory              Mayur      Jitendra      Ranjan      Alam Khan      Ahsaas      Revathi      Urvi Singh
                        More      Kumar      Raj
Midnight Mass             Kate      Zach      Hamish      Henry      Kristin      Samantha      Igby Rigney
                        Siegel      Gilford      Linklater      Thomas      Lehman      Sloyan
...           ...           ...           ...           ...           ...           ...           ..
Zinzana                  Ali      Saleh      Yasa      Ali Al-Jabri      Mansoor      Ahd      None
                        Suliman      Bakri
Zodiac                   Mark      Jake      Robert      Anthony      Brian      Elias      Dona
                        Ruffalo      Gyllenhaal      Downey Jr.      Edwards      Cox      Koteas      Logue
Zombieland               Jesse      Woody      Emma      Abigail      Amber      Bill      Derek Grat
                        Eisenberg      Harrelson      Stone      Breslin      Heard      Murray
Zoom                     Tim      Courteney      Chevy      Kate      Ryan      Michael      Spencer
                        Allen      Cox      Chase      Mara      Newman      Cassidy      Breslin
Zubaan                   Vicky      Sarah-      Raaghav      Manish      Meghna      Malkeet      Anita
                        Kaushal      Jane Dias      Chanana      Chaudhary      Malik      Rauni      Shabdish

8525 rows × 50 columns
```



```
In [124...] df_cast = df_cast.stack()
df_cast
```

```
Out[124... title
Dick Johnson Is Dead  0          nan
Blood & Water        0      Ama Qamata
                     1      Khosi Ngema
                     2      Gail Mabalane
                     3      Thabang Molaba
                     ...
Zubaan               3      Manish Chaudhary
                     4      Meghna Malik
                     5      Malkeet Rauni
                     6      Anita Shabdish
                     7      Chittaranjan Tripathy
Length: 63644, dtype: object
```

```
In [125... df_cast = pd.DataFrame(df_cast)
df_cast
```

```
Out[125... 0

      title
-----
Dick Johnson Is Dead  0          nan
Blood & Water        0      Ama Qamata
                     1      Khosi Ngema
                     2      Gail Mabalane
                     3      Thabang Molaba
                     ...      ...
Zubaan               3      Manish Chaudhary
                     4      Meghna Malik
                     5      Malkeet Rauni
                     6      Anita Shabdish
                     7      Chittaranjan Tripathy
```

63644 rows × 1 columns

```
In [126... df_cast.reset_index(inplace=True)
df_cast
```

Out[126...

	title	level_1	0
0	Dick Johnson Is Dead	0	nan
1	Blood & Water	0	Ama Qamata
2	Blood & Water	1	Khosi Ngema
3	Blood & Water	2	Gail Mabalane
4	Blood & Water	3	Thabang Molaba
...	...	...	...
63639	Zubaan	3	Manish Chaudhary
63640	Zubaan	4	Meghna Malik
63641	Zubaan	5	Malkeet Rauni
63642	Zubaan	6	Anita Shabdish
63643	Zubaan	7	Chittaranjan Tripathy

63644 rows × 3 columns

In [127...

```
df_cast = df_cast[['title',0]]
df_cast
```

Out[127...

	title	0
0	Dick Johnson Is Dead	nan
1	Blood & Water	Ama Qamata
2	Blood & Water	Khosi Ngema
3	Blood & Water	Gail Mabalane
4	Blood & Water	Thabang Molaba
...	...	...
63639	Zubaan	Manish Chaudhary
63640	Zubaan	Meghna Malik
63641	Zubaan	Malkeet Rauni
63642	Zubaan	Anita Shabdish
63643	Zubaan	Chittaranjan Tripathy

63644 rows × 2 columns

In [128...

```
df_cast.columns=['title', 'cast']
df_cast
```

Out[128...

	title	cast
0	Dick Johnson Is Dead	nan
1	Blood & Water	Ama Qamata
2	Blood & Water	Khosi Ngema
3	Blood & Water	Gail Mababane
4	Blood & Water	Thabang Molaba
...	...	...
63639	Zubaan	Manish Chaudhary
63640	Zubaan	Meghna Malik
63641	Zubaan	Malkeet Rauni
63642	Zubaan	Anita Shabdish
63643	Zubaan	Chittaranjan Tripathy

63644 rows × 2 columns

In [129...

```
df_cast.replace('nan', np.nan, inplace=True)
df_cast.isna().sum()
```

C:\Users\DELL\AppData\Local\Temp\ipykernel_7900\3796400136.py:1: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df_cast.replace('nan', np.nan, inplace=True)
```

Out[129...

```
title    0
cast    704
dtype: int64
```

Now we will split all the nested data individually and save them in a new dataframe, and then we will merge all of them so get the final dataframe.

In the process we will lose the format of null values from np.NaN to string NaN, so that requires one additional step which is done in the last 2 lines of each block We can confirm the count of null values from the above block

In [130...

```
df_genre = pd.DataFrame(df['genre'].apply(lambda x: str(x).split(',')).tolist(),in
df_genre = df_genre.stack()
df_genre = pd.DataFrame(df_genre)
df_genre.reset_index(inplace=True)
df_genre = df_genre[['title',0]]
df_genre.columns = ['title','genre']
df_genre.replace('nan', np.nan, inplace=True)
df_genre.isna().sum()
```

```
Out[130... title    0
genre    0
dtype: int64
```

```
In [131... df.columns
```

```
Out[131... Index(['type', 'title', 'director', 'cast', 'country', 'date_added',
      'release_year', 'rating', 'duration', 'genre', 'description', 'day',
      'month', 'year', 'month_name', 'weekday'],
      dtype='object')
```

```
In [132... df_director = pd.DataFrame(df['director'].apply(lambda x: str(x).split(',')).tolist()
df_director = df_director.stack()
df_director = pd.DataFrame(df_director)
df_director.reset_index(inplace=True)
df_director = df_director[['title',0]]
df_director.columns = ['title','director']
df_director.replace('nan',np.nan,inplace=True)
df_director.isna().sum()
```

```
Out[132... title    0
director  2439
dtype: int64
```

```
In [133... df_country = pd.DataFrame(df['country'].apply(lambda x: str(x).split(',')).tolist()
df_country = df_country.stack()
df_country = pd.DataFrame(df_country)
df_country.reset_index(inplace=True)
df_country = df_country[['title',0]]
df_country.columns = ['title','country']
df_country.replace('nan',np.nan,inplace=True)
df_country.isna().sum()
```

```
Out[133... title    0
country  730
dtype: int64
```

```
In [134... df12 = df_cast.merge(df_genre, on = 'title')
df12
```


Out[134...

	title	cast	genre
0	Dick Johnson Is Dead	NaN	Documentaries
1	Blood & Water	Ama Qamata	International TV Shows
2	Blood & Water	Ama Qamata	TV Dramas
3	Blood & Water	Ama Qamata	TV Mysteries
4	Blood & Water	Khosi Ngema	International TV Shows
...	...	...	...
146662	Zubaan	Anita Shabdish	International Movies
146663	Zubaan	Anita Shabdish	Music & Musicals
146664	Zubaan	Chittaranjan Tripathy	Dramas
146665	Zubaan	Chittaranjan Tripathy	International Movies
146666	Zubaan	Chittaranjan Tripathy	Music & Musicals

146667 rows × 3 columns

In [135...

```
df123 = df_director.merge(df12, on = 'title')
df123
```

Out[135...

	title	director	cast	genre
0	Dick Johnson Is Dead	Kirsten Johnson	NaN	Documentaries
1	Blood & Water	NaN	Ama Qamata	International TV Shows
2	Blood & Water	NaN	Ama Qamata	TV Dramas
3	Blood & Water	NaN	Ama Qamata	TV Mysteries
4	Blood & Water	NaN	Khosi Ngema	International TV Shows
...	...	...	...	...
158309	Zubaan	Mozez Singh	Anita Shabdish	International Movies
158310	Zubaan	Mozez Singh	Anita Shabdish	Music & Musicals
158311	Zubaan	Mozez Singh	Chittaranjan Tripathy	Dramas
158312	Zubaan	Mozez Singh	Chittaranjan Tripathy	International Movies
158313	Zubaan	Mozez Singh	Chittaranjan Tripathy	Music & Musicals

158314 rows × 4 columns

In [136...

```
df1234 = df_country.merge(df123, on = 'title')
df1234
```

Out[136...

	title	country	director	cast	genre
0	Dick Johnson Is Dead	United States	Kirsten Johnson	NaN	Documentaries
1	Blood & Water	South Africa	NaN	Ama Qamata	International TV Shows
2	Blood & Water	South Africa	NaN	Ama Qamata	TV Dramas
3	Blood & Water	South Africa	NaN	Ama Qamata	TV Mysteries
4	Blood & Water	South Africa	NaN	Khosi Ngema	International TV Shows
...	...	...	...	...	...
198399	Zubaan	India	Mozes Singh	Anita Shabdish	International Movies
198400	Zubaan	India	Mozes Singh	Anita Shabdish	Music & Musicals
198401	Zubaan	India	Mozes Singh	Chittaranjan Tripathy	Dramas
198402	Zubaan	India	Mozes Singh	Chittaranjan Tripathy	International Movies
198403	Zubaan	India	Mozes Singh	Chittaranjan Tripathy	Music & Musicals

198404 rows × 5 columns

In [137...

```
df_new = df.merge(df1234,on="title")
df_new
```

Out[137...

	type	title	director_x	cast_x	country_x	date_added	release_year	rating
0	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	2021-09-25	2020	Teens
1	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	2021-09-24	2021	Adults
2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	2021-09-24	2021	Adults
3	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	2021-09-24	2021	Adults
4	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	2021-09-24	2021	Adults
...	...	...	...	...	...	...	...	...
198399	Movie	Zubaan	Mozez Singh	Vicky Kaushal, Sarah-Jane Dias, Raaghav Chanan...	India	2019-03-02	2015	Teens
198400	Movie	Zubaan	Mozez Singh	Vicky Kaushal, Sarah-Jane Dias, Raaghav Chanan...	India	2019-03-02	2015	Teens

	type	title	director_x	cast_x	country_x	date_added	release_year	rating
198401	Movie	Zubaan	Mozez Singh	Vicky Kaushal, Sarah-Jane Dias, Raaghav Chanan...	India	2019-03-02	2015	Teens
198402	Movie	Zubaan	Mozez Singh	Vicky Kaushal, Sarah-Jane Dias, Raaghav Chanan...	India	2019-03-02	2015	Teens
198403	Movie	Zubaan	Mozez Singh	Vicky Kaushal, Sarah-Jane Dias, Raaghav Chanan...	India	2019-03-02	2015	Teens

198404 rows × 20 columns

In [138...

```
df_new.drop(columns = ['cast_x', 'country_x', 'director_x', 'genre_x'], inplace=True)
df_new
```

Out[138...

	type	title	date_added	release_year	rating	duration	description	day	moi
0	Movie	Dick Johnson Is Dead	2021-09-25	2020	Teens	90	As her father nears the end of his life, filmm...	25	
1	TV Show	Blood & Water	2021-09-24	2021	Adults	2	After crossing paths at a party, a Cape Town t...	24	
2	TV Show	Blood & Water	2021-09-24	2021	Adults	2	After crossing paths at a party, a Cape Town t...	24	
3	TV Show	Blood & Water	2021-09-24	2021	Adults	2	After crossing paths at a party, a Cape Town t...	24	
4	TV Show	Blood & Water	2021-09-24	2021	Adults	2	After crossing paths at a party, a Cape Town t...	24	
...	...	...	...	...	...	...	...	...	...
198399	Movie	Zubaan	2019-03-02	2015	Teens	111	A scrappy but poor boy worms his way into a ty...	2	
198400	Movie	Zubaan	2019-03-02	2015	Teens	111	A scrappy but poor boy worms his way into a ty...	2	
198401	Movie	Zubaan	2019-03-02	2015	Teens	111	A scrappy but poor boy worms his way into a ty...	2	
198402	Movie	Zubaan	2019-03-02	2015	Teens	111	A scrappy but poor	2	

	type	title	date_added	release_year	rating	duration	description	day	moi
							boy worms his way into a ty...		
198403	Movie	Zubaan	2019-03-02	2015	Teens	111	A scrappy but poor boy worms his way into a ty...	2	

198404 rows × 16 columns

```
In [139... df_new.rename(columns={'country_y':'country','director_y':'director',  
                        'cast_y':'cast','genre_y':'genre'},inplace=True)
```

```
In [140... df_new.head()
```

Out[140...

	type	title	date_added	release_year	rating	duration	description	day	month	y
0	Movie	Dick Johnson Is Dead	2021-09-25	2020	Teens	90	As her father nears the end of his life, filmm...	25	9	2
1	TV Show	Blood & Water	2021-09-24	2021	Adults	2	After crossing paths at a party, a Cape Town t...	24	9	2
2	TV Show	Blood & Water	2021-09-24	2021	Adults	2	After crossing paths at a party, a Cape Town t...	24	9	2
3	TV Show	Blood & Water	2021-09-24	2021	Adults	2	After crossing paths at a party, a Cape Town t...	24	9	2
4	TV Show	Blood & Water	2021-09-24	2021	Adults	2	After crossing paths at a party, a Cape Town t...	24	9	2

In [141...

```
df_new.shape
```

Out[141...

(198404, 16)

This is the new dataframe which has 199950 rows and 16 columns after unnesting the data

In [142...

```
df_new.isnull().sum()
```

```
Out[142... type          0
          title         0
          date_added     0
          release_year   0
          rating         0
          duration       0
          description     0
          day            0
          month          0
          year           0
          month_name     0
          weekday        0
          country        11699
          director       48645
          cast           1882
          genre          0
          dtype: int64
```

Here we will not delete the missing values as it will affect the data and other columns will get manipulated, hence we will replace them with a global value 'unknown'

```
In [143... df_new['cast']=df_new['cast'].replace(np.nan, 'Unknown')
df_new['country'] = df_new['country'].replace(np.nan, 'Unknown')
df_new['director'] = df_new['director'].replace(np.nan, 'Unknown')
```

```
In [144... df_new.head()
```


Out[144...]

	type	title	date_added	release_year	rating	duration	description	day	month	y
0	Movie	Dick Johnson Is Dead	2021-09-25	2020	Teens	90	As her father nears the end of his life, filmm...	25	9	2
1	TV Show	Blood & Water	2021-09-24	2021	Adults	2	After crossing paths at a party, a Cape Town t...	24	9	2
2	TV Show	Blood & Water	2021-09-24	2021	Adults	2	After crossing paths at a party, a Cape Town t...	24	9	2
3	TV Show	Blood & Water	2021-09-24	2021	Adults	2	After crossing paths at a party, a Cape Town t...	24	9	2
4	TV Show	Blood & Water	2021-09-24	2021	Adults	2	After crossing paths at a party, a Cape Town t...	24	9	2

In []:

In [145...]

```

null = df_new.isna().sum().sort_values(ascending=False)
percent = round(df_new.isna().mean()*100,2).sort_values(ascending = False)

missing_data = pd.concat([null,percent],axis=1,keys=['Missing Value Count','Percent

print('Total Records Present: ',df_new.shape[0])
print(missing_data[missing_data['Missing Value Count']!=0])
print('Total', ' ',missing_data['Missing Value Count'].sum(), ' ')

```

Total Records Present: 198404

Empty DataFrame

Columns: [Missing Value Count, Percentage]

Index: []

Total 0 0.0

After replacing we can confirm no more null values

In [146... `df_new.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 198404 entries, 0 to 198403
Data columns (total 16 columns):
#   Column          Non-Null Count  Dtype
---  -
0   type             198404 non-null  object
1   title            198404 non-null  object
2   date_added       198404 non-null  datetime64[ns]
3   release_year     198404 non-null  int64
4   rating           198404 non-null  object
5   duration         198404 non-null  int64
6   description      198404 non-null  object
7   day              198404 non-null  int32
8   month            198404 non-null  int32
9   year             198404 non-null  int32
10  month_name       198404 non-null  object
11  weekday          198404 non-null  object
12  country          198404 non-null  object
13  director         198404 non-null  object
14  cast             198404 non-null  object
15  genre            198404 non-null  object
dtypes: datetime64[ns](1), int32(3), int64(2), object(10)
memory usage: 21.9+ MB
```

In [147... `df_new.drop(["description"],axis=1,inplace=True)`In [148... `df_new.duplicated().sum()`Out[148... `np.int64(55)`

In the new dataframe, we can see 55 rows which are duplicated, so deleting them.

In [149... `df_new.drop_duplicates(keep='first',inplace=True)`

In []:

Creating 2 more dataframes for better analysis by dividing the present dataframe on the basis of type i.e Movie and TV Show and naming them df_movies and df_tvs.

```
In [150... df_movies = df_new[df_new['type']=='Movie']
df_tvs = df_new[df_new['type']=='TV Show']
```

In [151... `df_movies.shape`Out[151... `(144240, 15)`In [152... `df_tvs.shape`Out[152... `(54109, 15)`

The duration column has values such as 90 minutes for movies and 2 seasons for tv shows, so we will remove the word 'minutes' and 'seasons' in their respective dataframes.

Here, we are replacing the column duration with a better name season for tv shows only in df_tvs.

```
In [153... df_tvs.rename(columns={'duration':'seasons'},inplace=True)
```

C:\Users\DELL\AppData\Local\Temp\ipykernel_7900\3221938497.py:1: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df_tvs.rename(columns={'duration':'seasons'},inplace=True)
```

Finally, we have 4 dataframes, we will use all of these according to the analysis required ahead

df = Cleaned data before unnesting.

df_new = Cleaned data after unnesting.

df_movies = Cleaned data of type - movie after unnesting.

df_tvs = Cleaned data of type - TV Show after unnesting

We are ready with our dataframes, and also ready to begin our EDA!

Lets start with checking the time period of our data.

```
In [154... print(df['year'].min())
print(df['year'].max())
```

2008

2021

- The data lies between the year 2008 and 2021.

Now lets see the count of content available on Netflix.

```
In [155... df.shape[0]
```

Out[155... 8525

- The Netflix library has 8612 movies or shows to watch.

Now, lets see the count of movies and TV Shows individually.

```
In [156... print('Movie: ', df[df['type']=='Movie'].shape[0])
print('TV Show: ', df[df['type']=='TV Show'].shape[0])
```

Movie: 6031
TV Show: 2494

- There are a total of 6031 movies and 2581 TV shows to available.

Now, lets see how many directors are present in the data.

```
In [157... df_director['director'].nunique()
```

Out[157... 4921

- There are a total of 4925 directors present in the data

Now, lets see the count of actors present.

```
In [158... df_cast['cast'].nunique()
```

Out[158... 35892

- There are a total of 36148 actors present in the data.

Lets check in how many countries in the data distributed in.

```
In [159... df_country['country'].nunique()
```

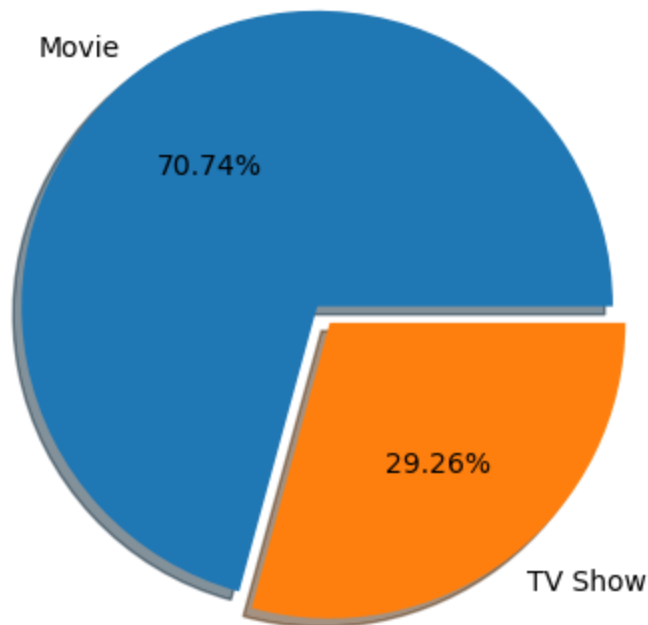
Out[159... 127

- The content is distributed across 127 countries.

Lets start our visualisation with seeing the distribution of content on Netflix. Here we will use df as we only need to count the row once.

```
In [160... import matplotlib.pyplot as plt
plt.pie(df["type"].value_counts(), labels=df["type"].value_counts().index,
        autopct="%1.2f%%", explode=[0.03, 0.04], shadow=True)
plt.suptitle('Movies vs TV Shows distribution')
plt.show()
```

Movies vs TV Shows distribution



Inference

- Netflix has 70% of its content as movies.
- TV Shows are clearly lesser than Movies.

Recommendations

- More TV Shows should be added as they create more suspense and have more story than a 100 minute movie.
- Users like to binge watch a particular story for a longer time.

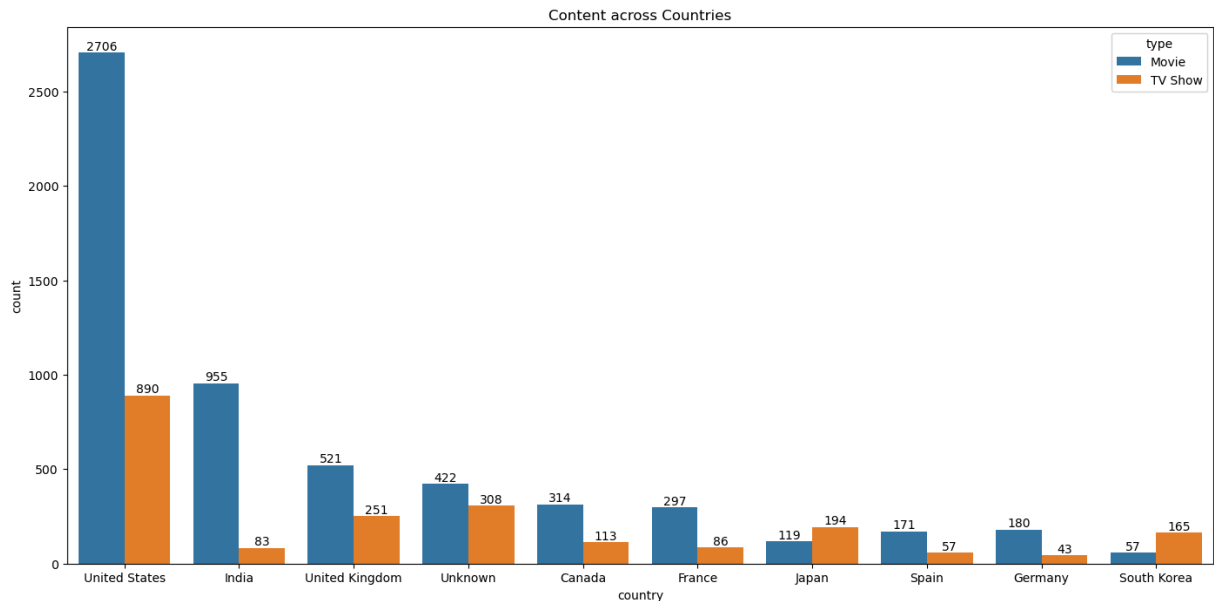
Lets see the content distribution across countries.

Here we need the unnested data for countries which is in `df_new` and we need to count the titles. After unnesting there is only one dataframe which has multiple rows because we also unnested the director, cast and genre, so hence we will do a `drop_duplicate` function on the `df_new` so that we will only get one row of the title and country group.

```
In [161... df_temp = df_new.drop_duplicates(subset = ['country', 'title'])
df_temp

x = df_temp['country'].value_counts().head(10)
x
```

```
plt.figure(figsize = (17,8))
plt.title('Content across Countries')
label = sns.countplot(data = df_temp, x = 'country', hue = 'type', order=x.index)
for i in label.containers:
    label.bar_label(i)
plt.show()
```



Inference

- US has the most content for movies followed by India and UK.
- TV Shows are mostly created in the US and UK.
- We can also observe that all the countries have more movies than TV Shows, whereas Japan and South Korea have more TV shows than movies.

Recommendation

- The difference between the number of movies and Tv shows for all the countries is very high especially for India.
- This should be minimised by adding more TV Shows as TV Shows can keep a user engaged for 2-3 seasons rated than a 100 minute movie.
- TV Shows have also been very popular in recent times and are the new and demanded versions of a movie.

Lets see the top 10 Directors.

Here we need unnested data of directors only so we will drop duplicates on director, title group in df_movies and df_tvs.

```
In [162... df_movies_temp = df_movies.drop_duplicates(subset = ['director','title'])
df_tvs_temp = df_tvs.drop_duplicates(subset = ['director','title'])

plt.figure(figsize = (17,7))
```

```

plt.suptitle('Top 10 Directors')

plt.subplot(1,2,1)
x=df_movies_temp['director'].value_counts()[1:11]
label = sns.countplot(data=df_movies_temp, x='director', order = x.index,palette="h
for i in label.containers:
    label.bar_label(i)
plt.title("Movies")
plt.xticks(rotation=45)
plt.xlabel('Directors')
plt.ylabel('Movie Count')

plt.subplot(1,2,2)
y=df_tvs_temp['director'].value_counts()[1:11]
label = sns.countplot(data=df_tvs_temp, x='director', order = y.index,palette="husl
for i in label.containers:
    label.bar_label(i)
plt.title("TV Shows")
plt.xticks(rotation=45)
plt.xlabel('Directors')
plt.ylabel('TV Show Count')
plt.show()

```

C:\Users\DELL\AppData\Local\Temp\ipykernel_7900\2509367928.py:9: FutureWarning:

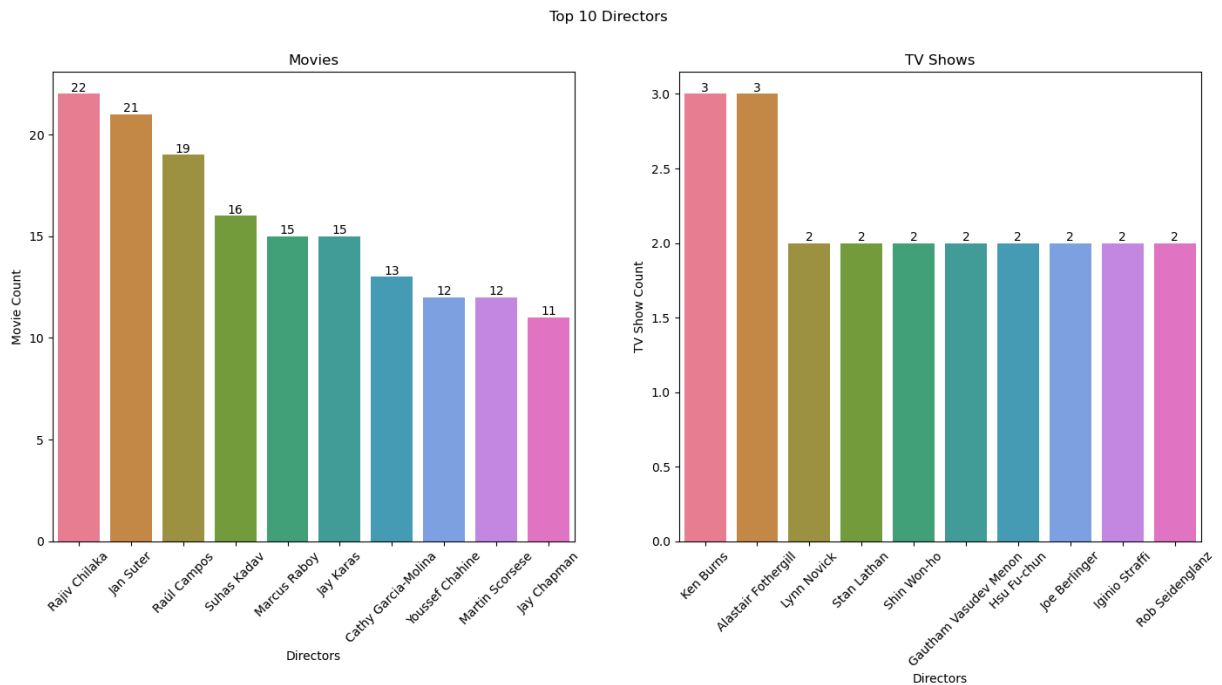
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
label = sns.countplot(data=df_movies_temp, x='director', order = x.index,palette="husl")
```

C:\Users\DELL\AppData\Local\Temp\ipykernel_7900\2509367928.py:19: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
label = sns.countplot(data=df_tvs_temp, x='director', order = y.index,palette="husl")
```



Inference

- Rajiv Chilaka, Jan Suter, Raul Campos are the most active directors with 22, 21 and 19 movies,
- Whereas for TV Shows all the directors have directed around 2-3 shows only.

Lets see the top 10 Genres

Here we need unnested data of genres only so we will drop duplicates on genre, title group in df_movies and df_tvs.

```
In [163... df_movies_temp = df_movies.drop_duplicates(subset = ['genre','title'])
df_tvs_temp = df_tvs.drop_duplicates(subset = ['genre','title'])

plt.figure(figsize = (17,7))
plt.suptitle('Top 10 Genres')

plt.subplot(1,2,1)
x=df_movies_temp['genre'].value_counts().head(10)
label = sns.countplot(data= df_movies_temp, x='genre',order = x.index,palette="husl")
for i in label.containers:
    label.bar_label(i)
plt.title("Movies")
plt.xticks(rotation=45)
plt.xlabel('Genres')
plt.ylabel('Movie Count')

plt.subplot(1,2,2)
y=df_tvs_temp['genre'].value_counts().head(10)
label = sns.countplot(data=df_tvs_temp, x='genre',
                      order = y.index,palette="viridis")
for i in label.containers:
```



```

    label.bar_label(i)
plt.title("TV Shows")
plt.xticks(rotation=45)
plt.xlabel('Genres')
plt.ylabel('TV Show Count')
plt.show()

```

C:\Users\DELL\AppData\Local\Temp\ipykernel_7900\4283709614.py:9: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```

label = sns.countplot(data= df_movies_temp, x='genre',order = x.index,palette="hus
1")

```

C:\Users\DELL\AppData\Local\Temp\ipykernel_7900\4283709614.py:19: FutureWarning:

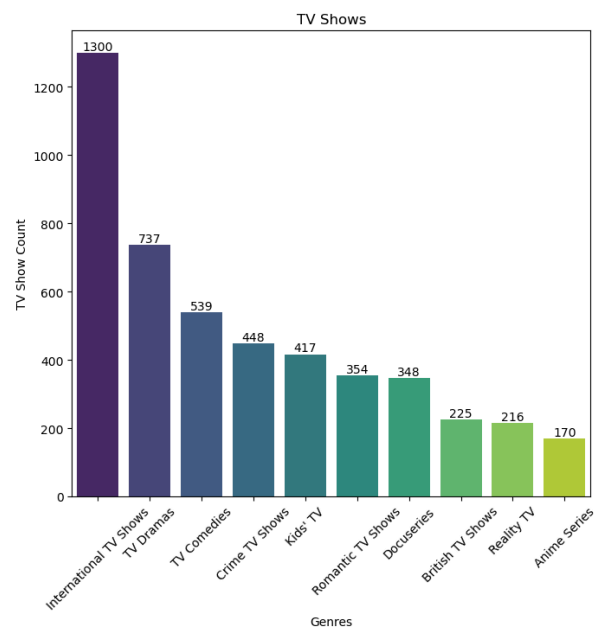
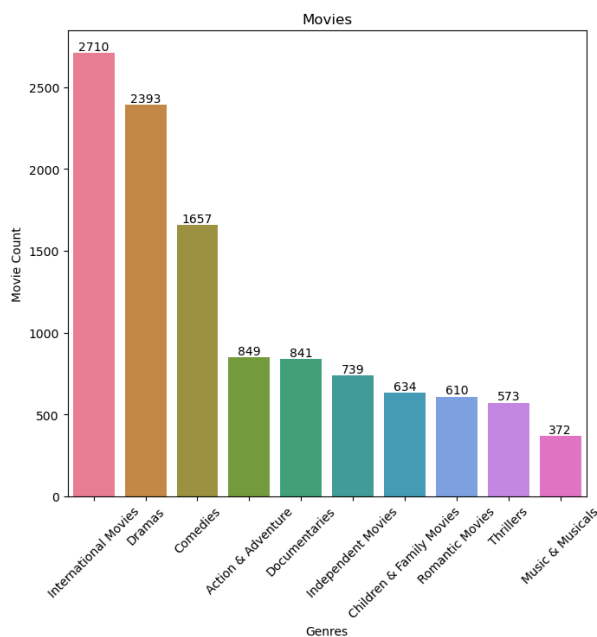
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```

label = sns.countplot(data=df_tvs_temp, x='genre',

```

Top 10 Genres



Inference

- From the above graph, it is inferred that most of the content fall under International followed by Drama and Comedy genre

Recommendations

- The count of International genre is very high as compared to other genres for TV Shows,
- Netflix should try adding more content of different genres as well.

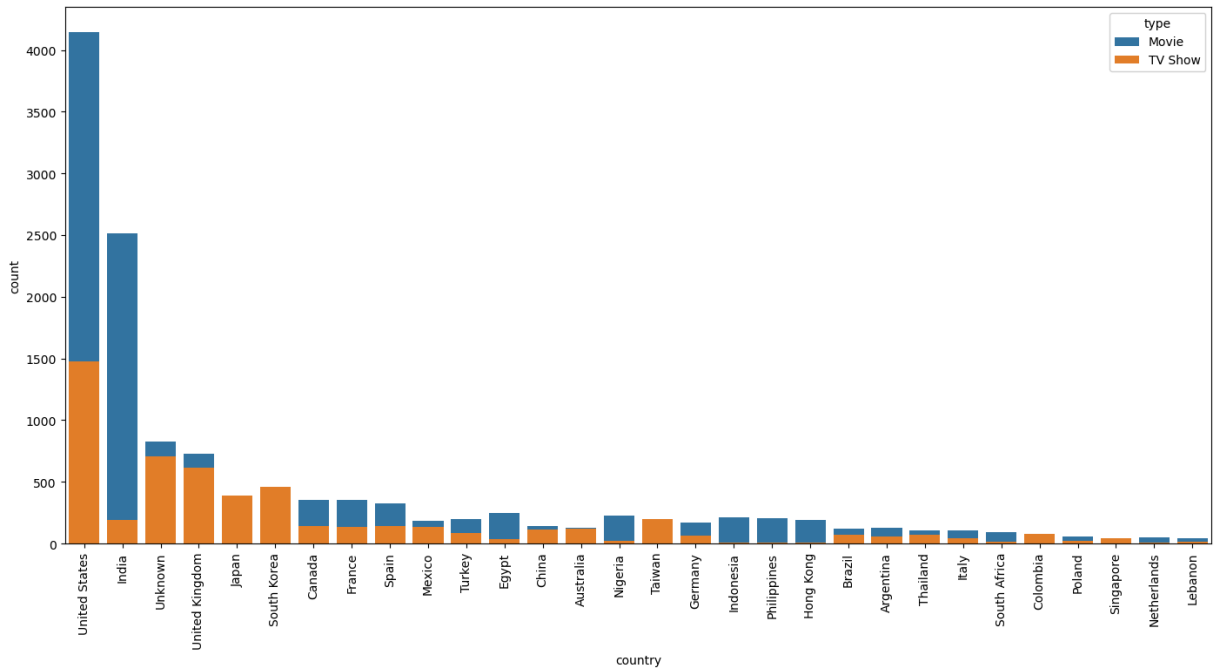
```

In [164... df_temp = df_new.drop_duplicates(subset = ['genre','title'])

plt.figure(figsize = (17,8))

```

```
plt.xticks(rotation=90)
sns.countplot(data = df_temp, x='country', hue = 'type', dodge = False,
              order = df_temp['country'].value_counts().head(30).index)
plt.show()
```



Lets see the content available age group wise

In [165...

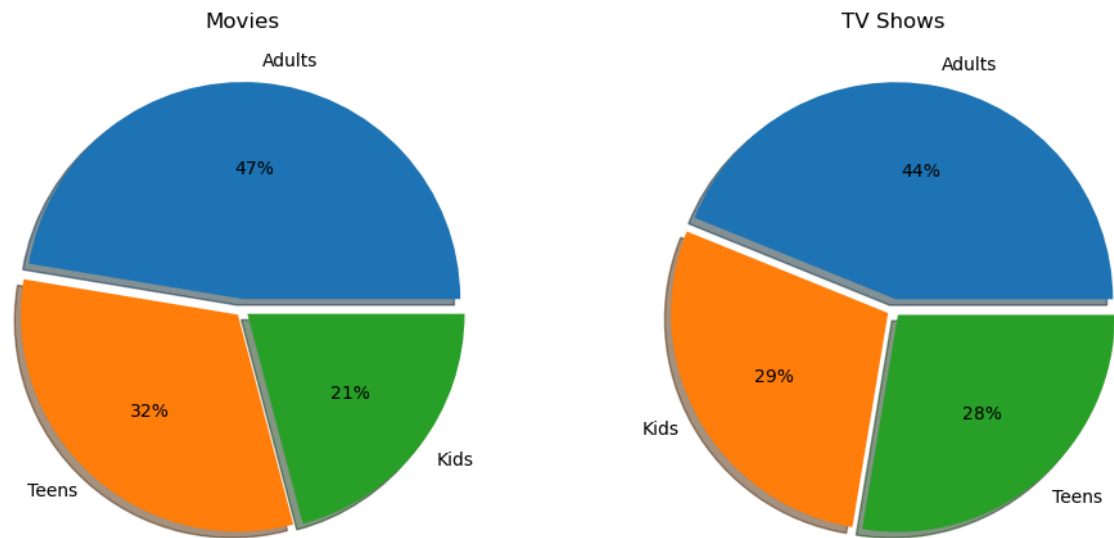
```
df_movies_temp = df_movies.drop_duplicates(subset = ['rating','title'])
df_tvs_temp = df_tvs.drop_duplicates(subset = ['rating','title'])

plt.figure(figsize = (12,6))
plt.suptitle('Classification of Content on Netflix')

plt.subplot(1,2,1)
plt.pie(df_movies_temp['rating'].value_counts(),
        labels = df_movies_temp['rating'].value_counts().index,
        autopct='%0.0f%%', explode = (0.05,0.03,0.03), shadow = True)
plt.title('Movies')

plt.subplot(1,2,2)
plt.pie(df_tvs_temp['rating'].value_counts(),
        labels = df_tvs_temp['rating'].value_counts().index,
        autopct='%0.0f%%', explode = (0.05,0.03,0.03), shadow = True)
plt.title('TV Shows')
plt.show()
```

Classification of Content on Netflix



```
In [166...] df_movies_temp['rating'].value_counts()
```

```
Out[166...] rating
Adults      2858
Teens       1913
Kids        1260
Name: count, dtype: int64
```

Inference

- Most content on Netflix is for the adults followed by Teens and Kids.

Recommendation

- More content should be added for the teens so as to increase the viewership.

Now, lets see when does netflix add movies and tv shows the most

```
In [167...] df_movies_temp = df_movies.drop_duplicates(subset = ['year','title'])
df_movies_temp

mv_year = df_movies_temp['year'].value_counts()
mv_year.sort_index(inplace=True)
mv_year

month_order = ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August',
               'October', 'November', 'December']
mv_month = df_movies_temp['month_name'].value_counts().loc[month_order]
mv_month

day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
mv_day = df_movies_temp['weekday'].value_counts().loc[day_order]
```

```

mv_day

plt.figure(figsize=(17,8))
plt.suptitle('Movies added on Netflix')

plt.subplot(1,3,1)
label = sns.countplot(data=df_movies_temp, x='year', order = mv_year.index,palette=
for i in label.containers:
    label.bar_label(i)
plt.xticks(rotation=45)
plt.xlabel('Year')
plt.title('Year wise')

plt.subplot(1,3,2)
label = sns.countplot(data=df_movies_temp, x='month_name',
                      order = mv_month.index,palette="husl")
for i in label.containers:
    label.bar_label(i)
plt.xticks(rotation=45)
plt.xlabel('Month')
plt.title('Month wise')

plt.subplot(1,3,3)
label = sns.countplot(data=df_movies_temp, x='weekday',
                      order = mv_day.index,palette="husl")
for i in label.containers:
    label.bar_label(i)
plt.xticks(rotation=45)
plt.xlabel('Weekday')
plt.title('Day wise')
plt.show()

```

C:\Users\DELL\AppData\Local\Temp\ipykernel_7900\3341213338.py:23: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
label = sns.countplot(data=df_movies_temp, x='year', order = mv_year.index,palette="viridis")
```

C:\Users\DELL\AppData\Local\Temp\ipykernel_7900\3341213338.py:31: FutureWarning:

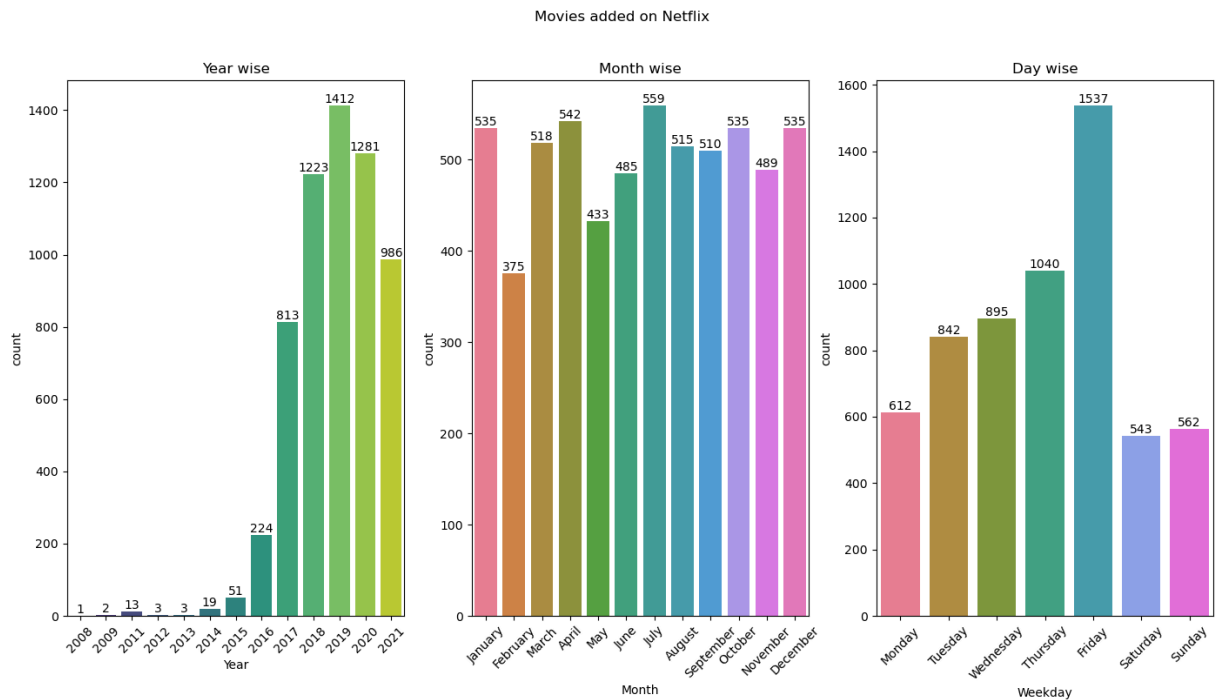
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
label = sns.countplot(data=df_movies_temp, x='month_name',
```

C:\Users\DELL\AppData\Local\Temp\ipykernel_7900\3341213338.py:40: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
label = sns.countplot(data=df_movies_temp, x='weekday',
```



In []:

Inference

- As per the data, most movies were released in the year 2019 and 2020.
- They were released the most in July.
- We can also see that most movies were released on Friday.

In [168...

```
df_tvs_temp = df_tvs.drop_duplicates(subset = ['year','title'])

tv_year = df_tvs_temp['year'].value_counts()
tv_year.sort_index(inplace=True)

month_order = ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September', 'October', 'November', 'December']
tv_month = df_tvs_temp['month_name'].value_counts().loc[month_order]

day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
tv_day = df_tvs_temp['weekday'].value_counts().loc[day_order]

plt.figure(figsize=(17,8))
plt.suptitle('TV Shows added on Netflix')

plt.subplot(1,3,1)
label = sns.countplot(data=df_tvs_temp, x='year', order = tv_year.index)
for i in label.containers:
    label.bar_label(i)
plt.xticks(rotation=45)
plt.xlabel('Year')
plt.title('Year wise')

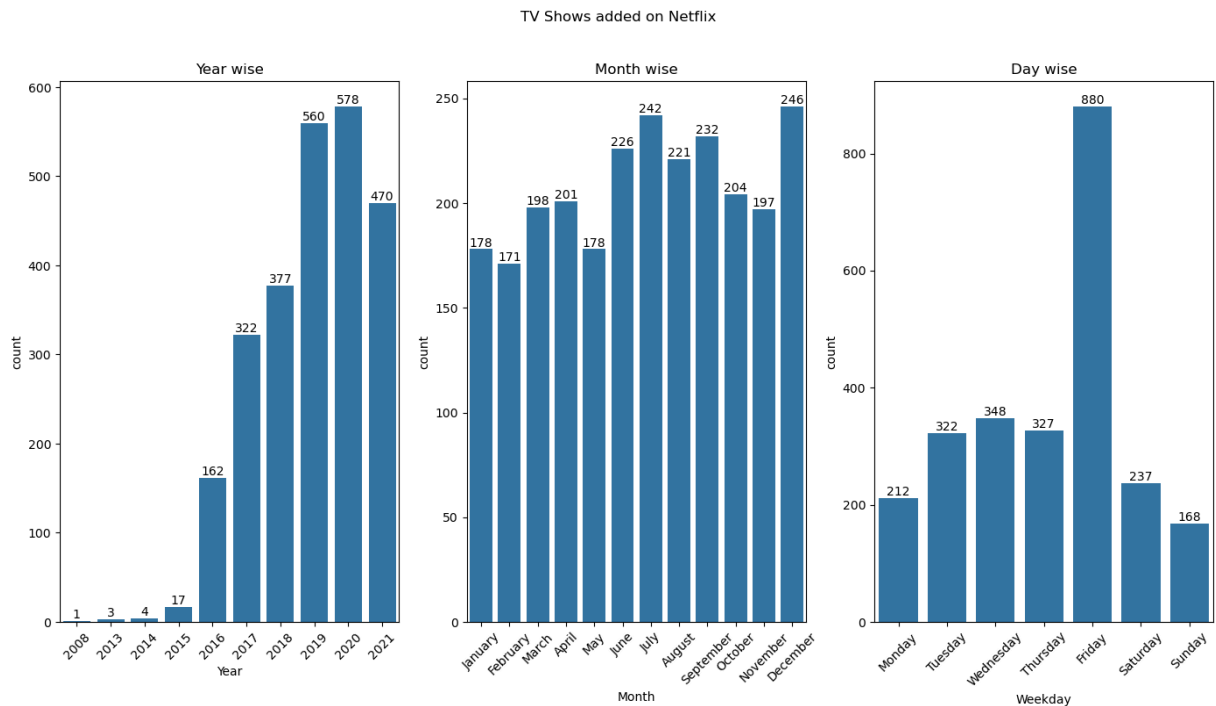
plt.subplot(1,3,2)
label = sns.countplot(data=df_tvs_temp, x='month_name', order = tv_month.index)
```

```

for i in label.containers:
    label.bar_label(i)
plt.xticks(rotation=45)
plt.xlabel('Month')
plt.title('Month wise')

plt.subplot(1,3,3)
label = sns.countplot(data=df_tvs_temp, x='weekday', order = tv_day.index)
for i in label.containers:
    label.bar_label(i)
plt.xticks(rotation=45)
plt.xlabel('Weekday')
plt.title('Day wise')
plt.show()

```



Inference

- As per the data, most TV Shows were released in the year 2019 and 2020.
- They were released the most in December followed by July and September.
- We can also see that most shows were released on Friday.

Recommendation

- To increase viewership in India, more shows should be released during the vacation or festival seasons which are around April-May and between October and December.

Lets see the average duration of a movie and a average season of TV Show

```

In [169... df_movies_temp = df_movies.drop_duplicates(subset = ['duration','title'])
df_tvs_temp = df_tvs.drop_duplicates(subset = ['seasons','title'])

plt.figure(figsize=(17,8))

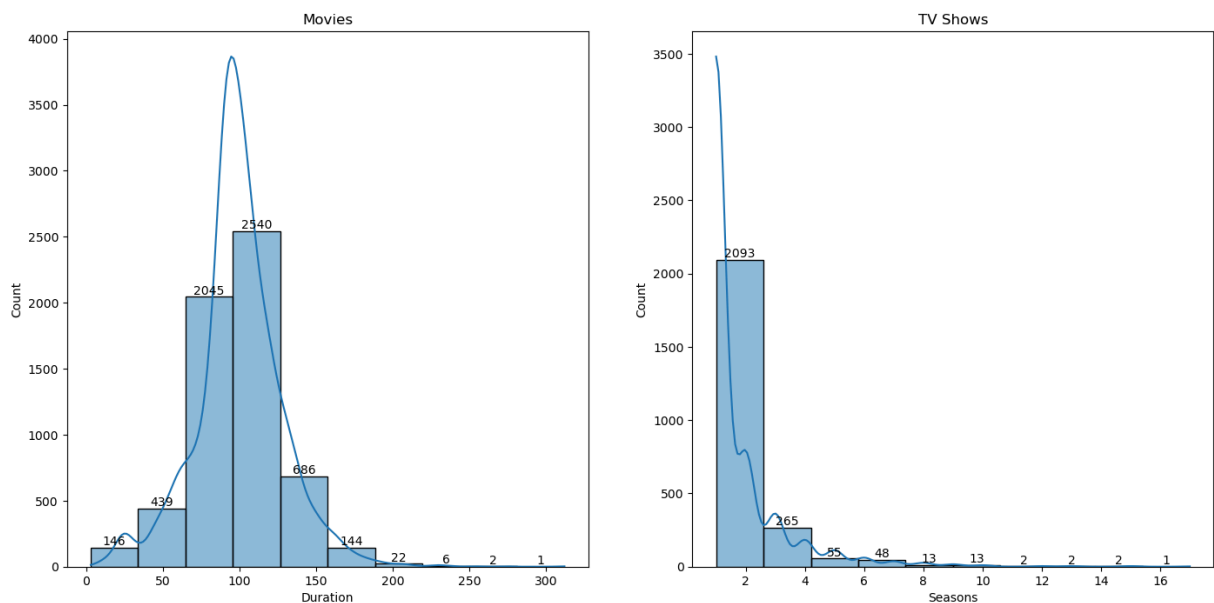
```

```
plt.suptitle('Average Duration and Seasons of Content on Netflix')

plt.subplot(1,2,1)
label = sns.histplot(df_movies_temp['duration'], bins=10, kde = True)
for i in label.containers:
    label.bar_label(i)
plt.xlabel('Duration')
plt.title('Movies')

plt.subplot(1,2,2)
label = sns.histplot(df_tvsv_temp['seasons'], bins=10, kde = True)
for i in label.containers:
    label.bar_label(i)
plt.xlabel('Seasons')
plt.title('TV Shows')
plt.show()
```

Average Duration and Seasons of Content on Netflix



Inference

- Most (Around 4500) movies have duration between 65 and 125 minutes.
- Most (Around 2200) TV Shows have been produced for around 2 seasons.

Recommendations

- Duration must be kept between 65 and 125 minutes for a movie.
- TV Show should have around 2-3 seasons

Lets see the range of most movie's duration and tv show's seasons lie.

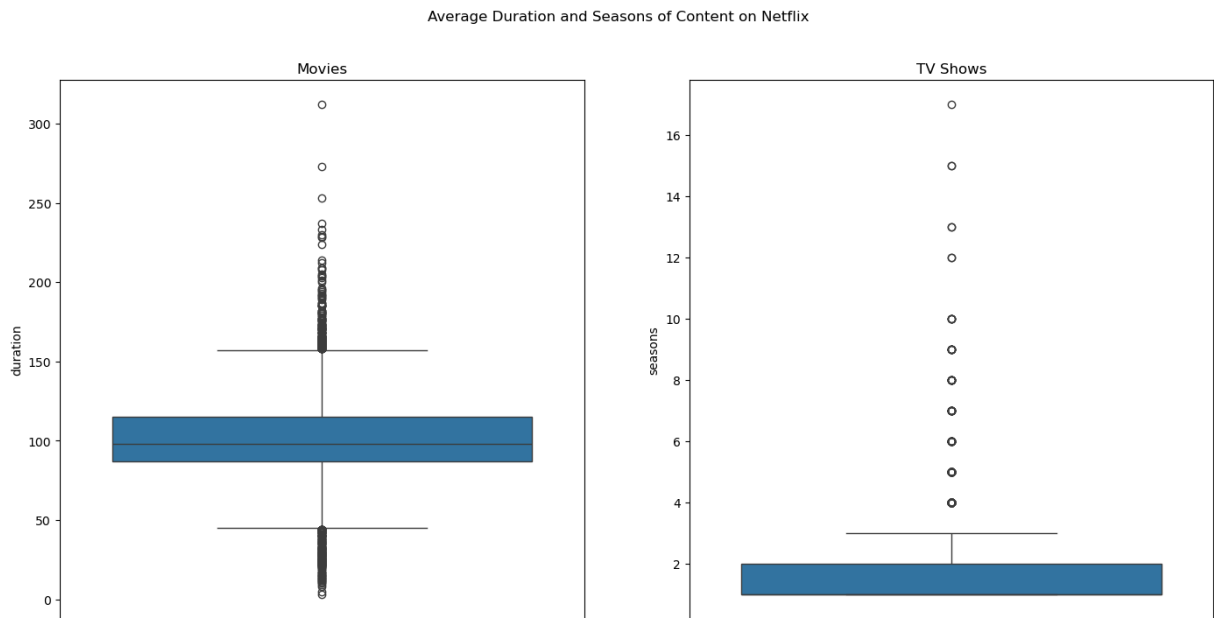
In [170...

```
plt.figure(figsize=(17,8))
plt.suptitle('Average Duration and Seasons of Content on Netflix')

plt.subplot(1,2,1)
sns.boxplot(y=df_movies_temp["duration"])
```

```
plt.title('Movies')

plt.subplot(1,2,2)
sns.boxplot(y=df_tvs_temp['seasons'])
plt.title('TV Shows')
plt.show()
```



Inference

- The median duration of a movie on Netflix is around 100 minutes, whereas the median season of a TV Show is 1 season.
- Most movies have duration length between 50 and 160 minutes.
- Most TV shows have either 1,2 or 3 seasons.

Recommendation

- In order to keep the audience engaged, it is recommended to keep the movie length upto 160 minutes and upto 3 seasons for a TV Show.

Lets see most active actors for movies and tv shows seperately.

```
In [171... df_movies_temp = df_movies.drop_duplicates(subset = ['cast','title'])
df_tvs_temp = df_tvs.drop_duplicates(subset = ['cast','title'])

plt.figure(figsize = (17,7))
plt.suptitle('Top 10 Actors')

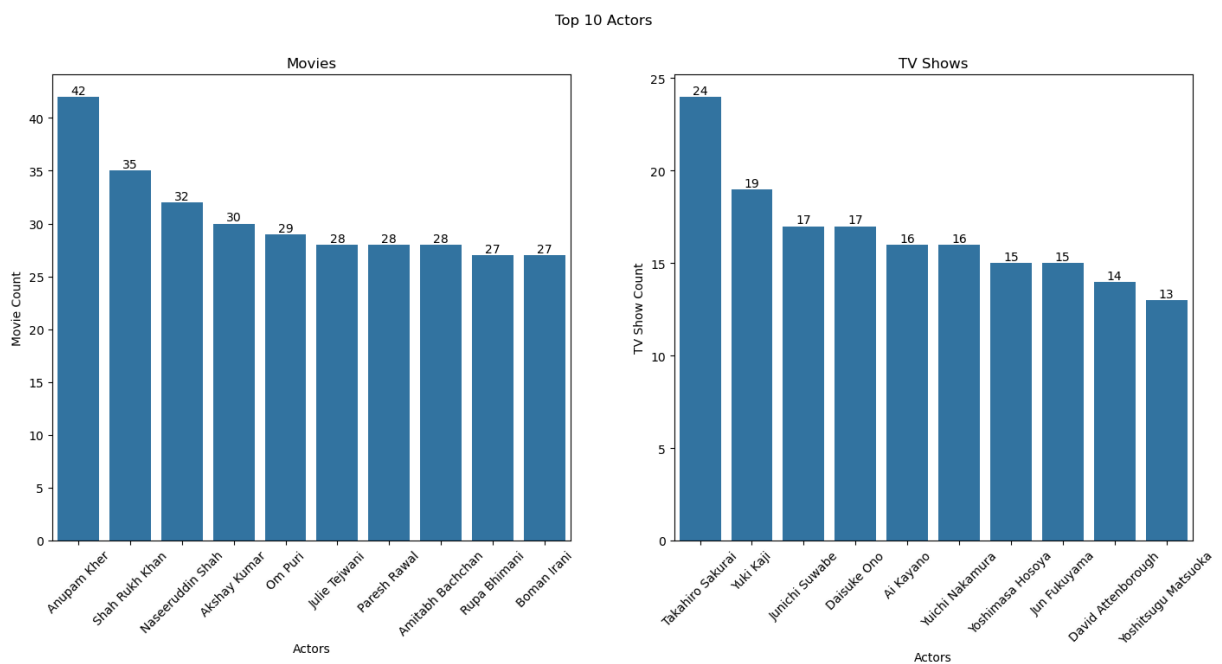
plt.subplot(1,2,1)
label = sns.countplot(data=df_movies_temp, x='cast',
                      order = df_movies_temp['cast'].value_counts()[1:11].index)
for i in label.containers:
    label.bar_label(i)
```



```
plt.title("Movies")
plt.xticks(rotation=45)
plt.xlabel('Actors')
plt.ylabel('Movie Count')

plt.subplot(1,2,2)
label = sns.countplot(data=df_tvs_temp, x='cast',
                      order = df_tvs_temp['cast'].value_counts()[1:11].index)
for i in label.containers:
    label.bar_label(i)
plt.title("TV Shows")
plt.xticks(rotation=45)
plt.xlabel('Actors')
plt.ylabel('TV Show Count')

plt.show()
```



Inference

- We can clearly see that Anupam Kher has done the most amount of movies followed by Shah Rukh Khan and Naseeruddin Shah.
- The top 10 actors for movies are all Indians, whereas the top 10 actors for TV Shows are from different countries.
- 8 out of the top 10 actors for movies are males and 2 are females.

Recommendations

- TV Shows have very few Indian actors as compared to other actors.
- To increase viewership of TV Shows in India, Netflix should add more TV Shows with Indian actors in it.

